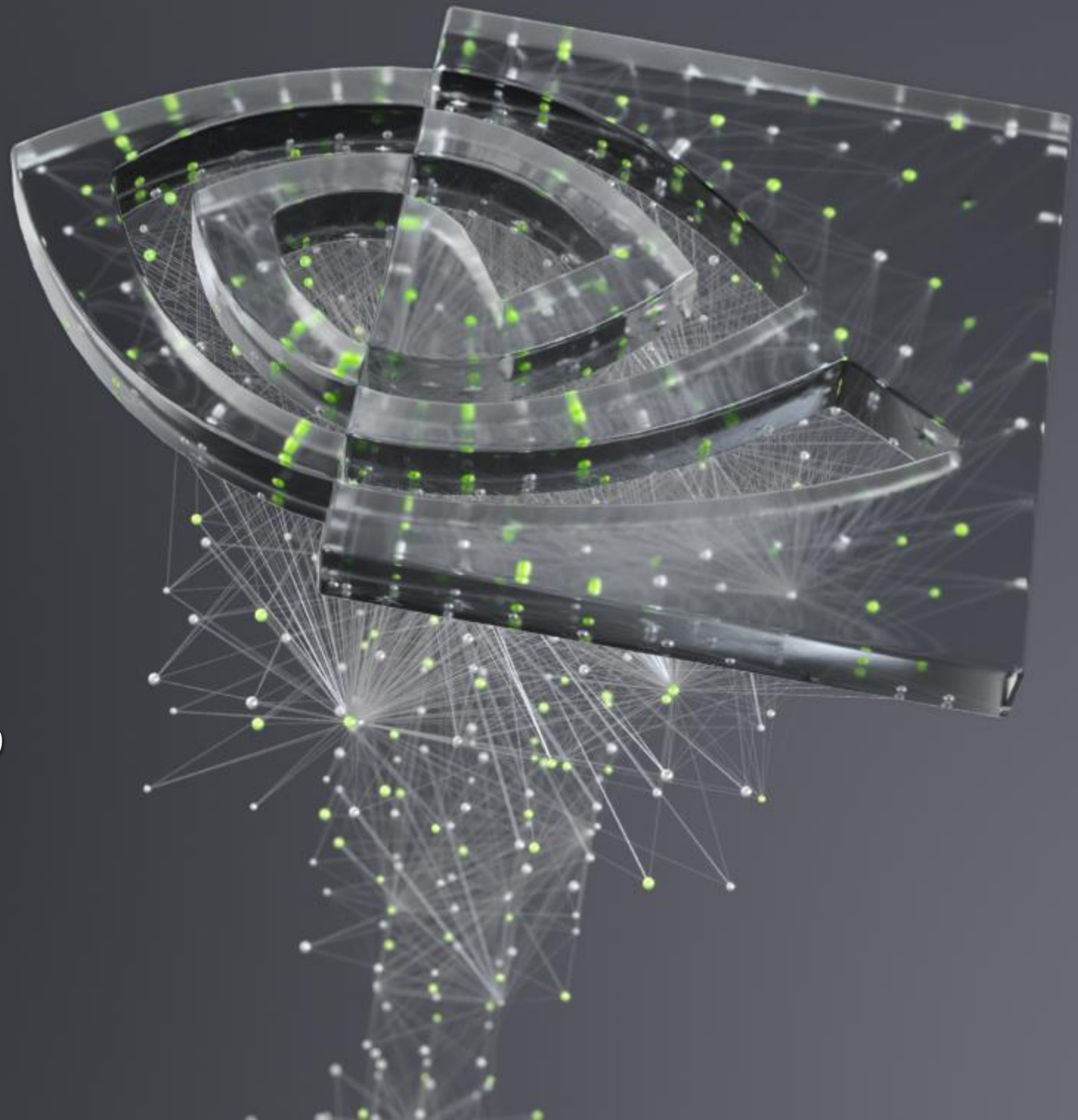


RESILIENT NEXTHOP GROUPS

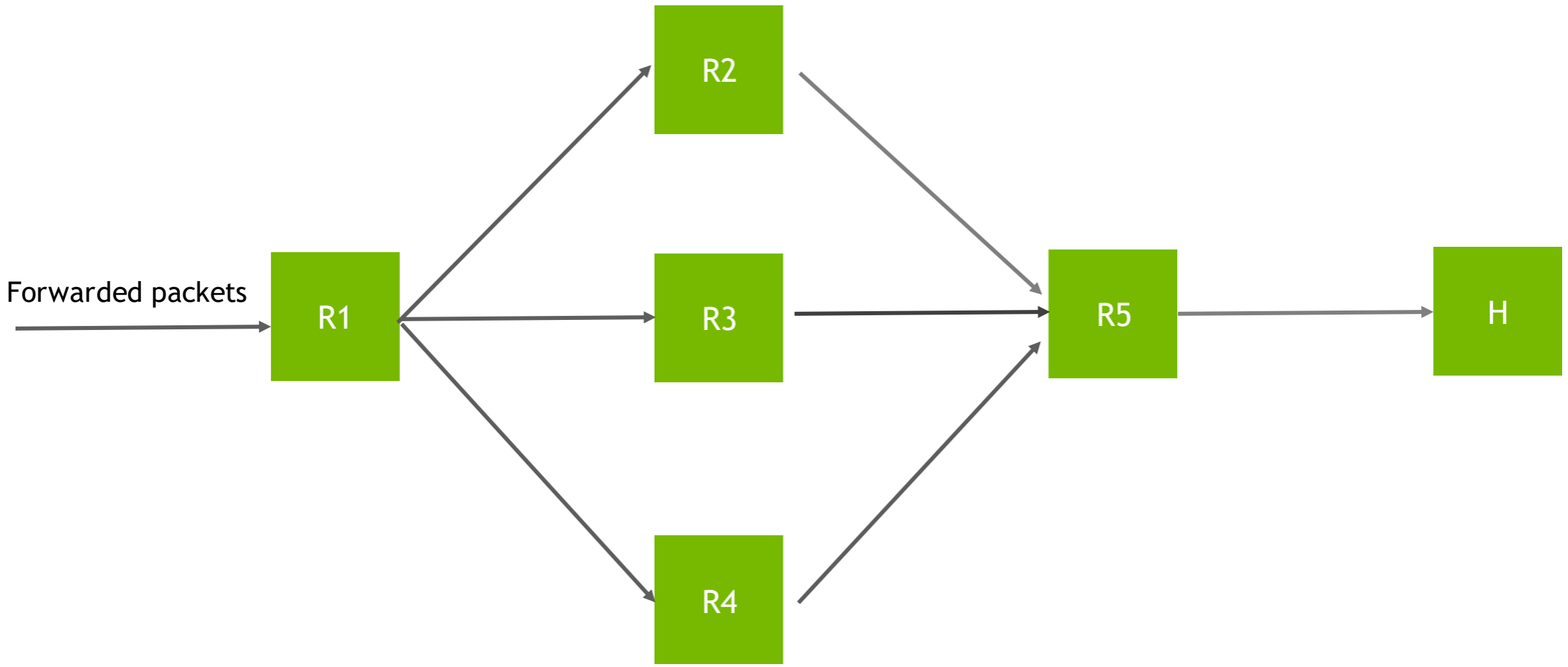
July 2021

AGENDA

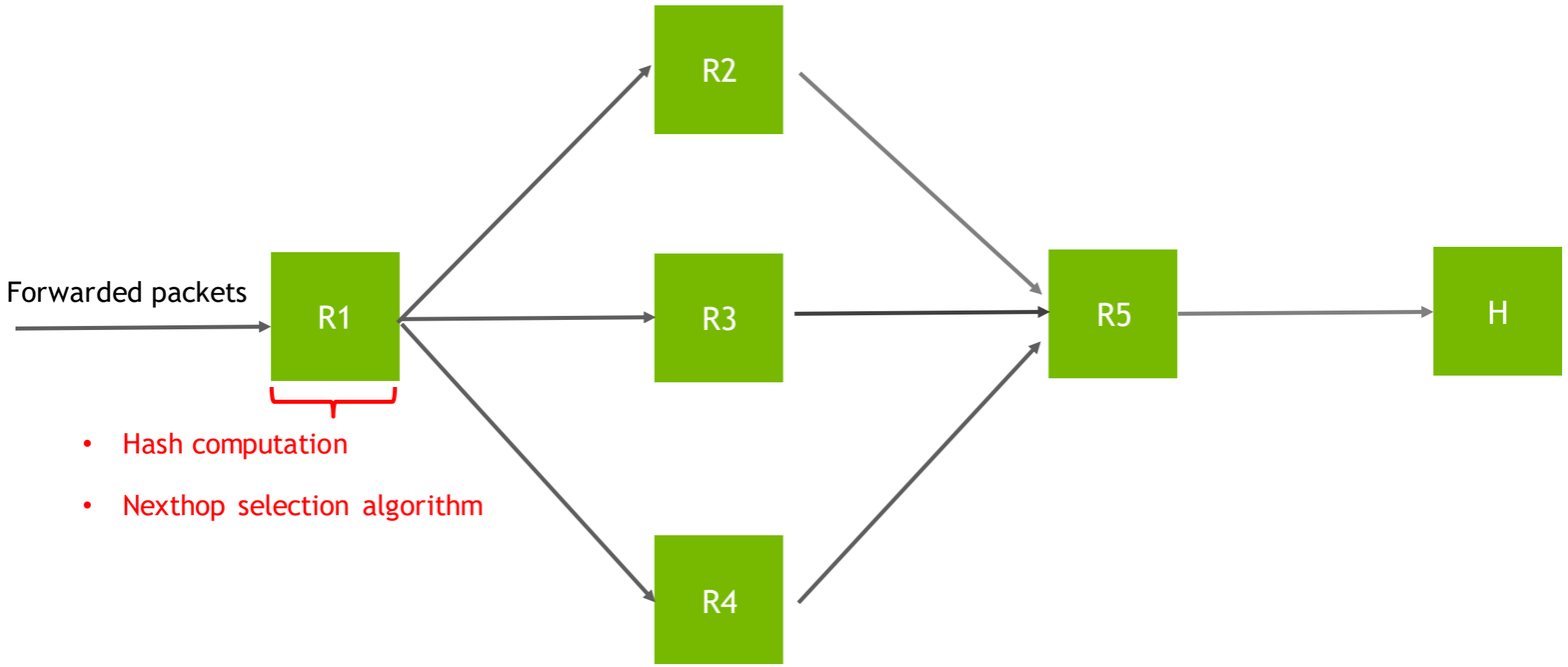
- Background
- Motivation
- Nexthop objects overview
- Resilient nexthop groups
  - Theory
  - Implementation
  - Hardware offload
  - Testing
  - Future extensions

NVIDIA.

BACKGROUND – MULTIPATH ROUTING



BACKGROUND – MULTIPATH ROUTING



BACKGROUND – MULTIPATH ROUTING

- **Hash calculation:** Based on which packet fields multipath hash is calculated. Controlled by net.ipv4.fib_multipath_hash_policy and net.ipv6.fib_multipath_hash_policy sysctls in Linux [1]
- **Nexthop selection:** Based on calculated hash, which nexthop to choose
  - **Hash-Threshold [2]:** Key space is divided into regions according to nexthops and their weights. Used by Linux for IPv4 [3] and IPv6 [4]

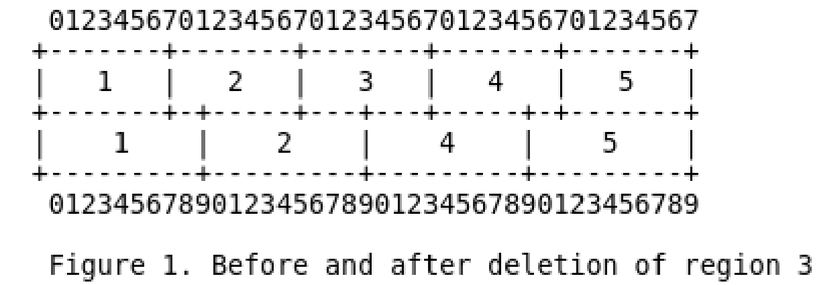


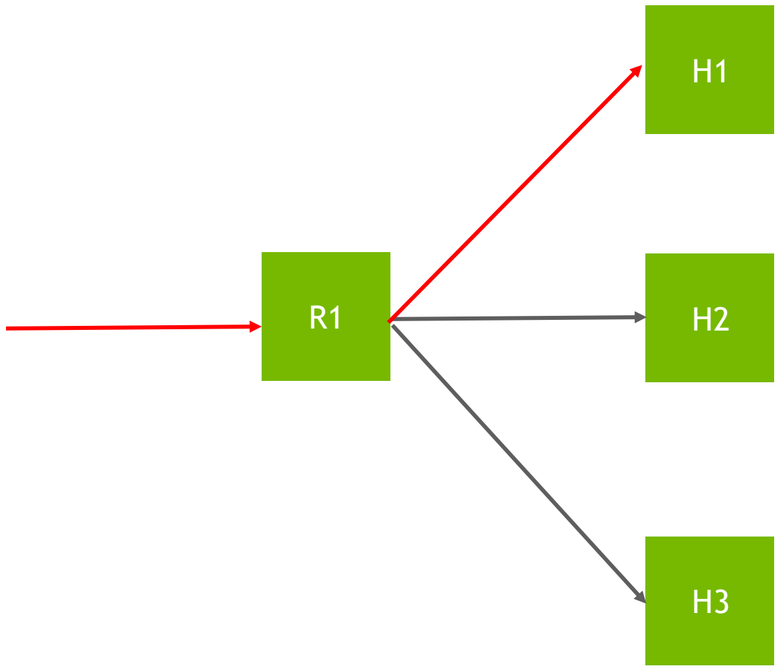
Figure 1. Before and after deletion of region 3

  - **Modulo-N [2]**

[1] https://www.kernel.org/doc/html/latest//networking/ip-sysctl.html
[2] https://datatracker.ietf.org/doc/html/rfc2992
[3] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=0e884c78ee19e902f300ed147083c28a0c6302f0
[4] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=3d709f69a3e749f4d1c195dab499df8ab66e25a8

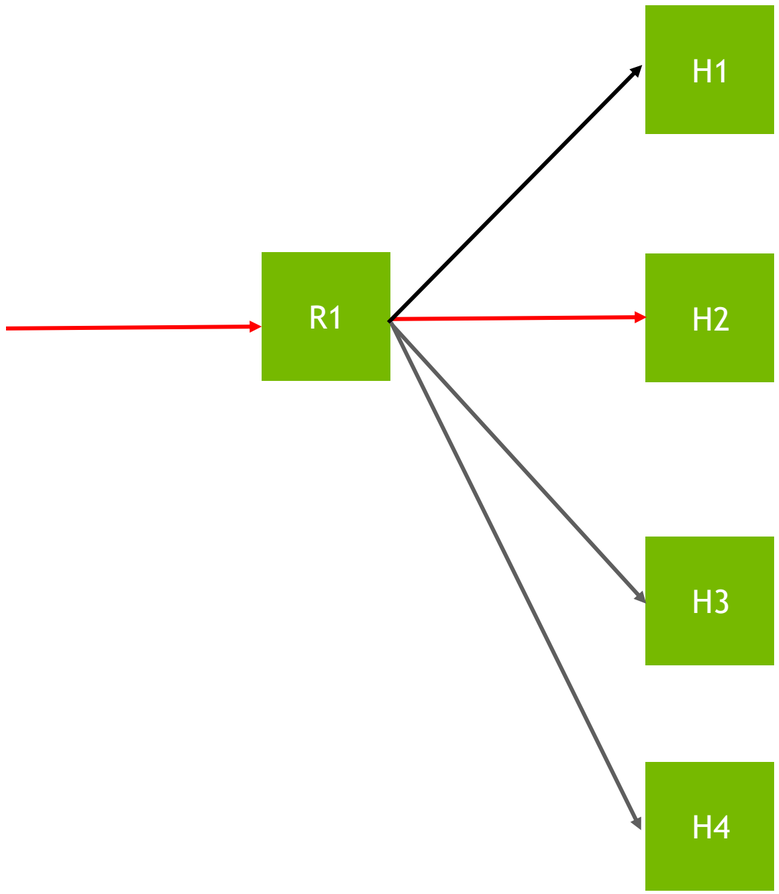
NVIDIA.

MOTIVATION – LOAD BALANCING

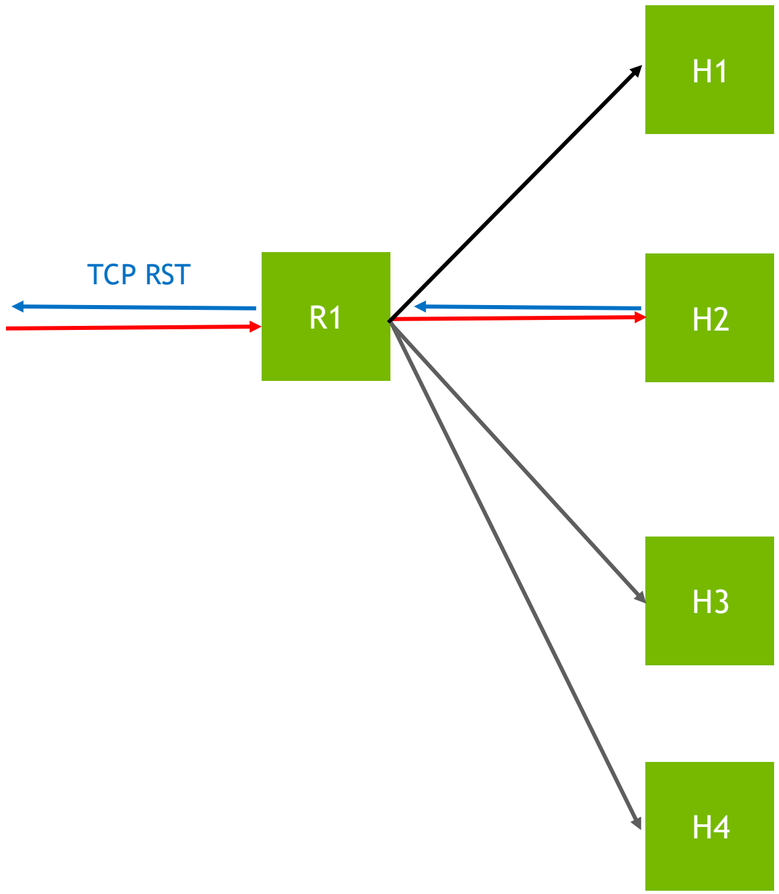


NVIDIA.

MOTIVATION – LOAD BALANCING

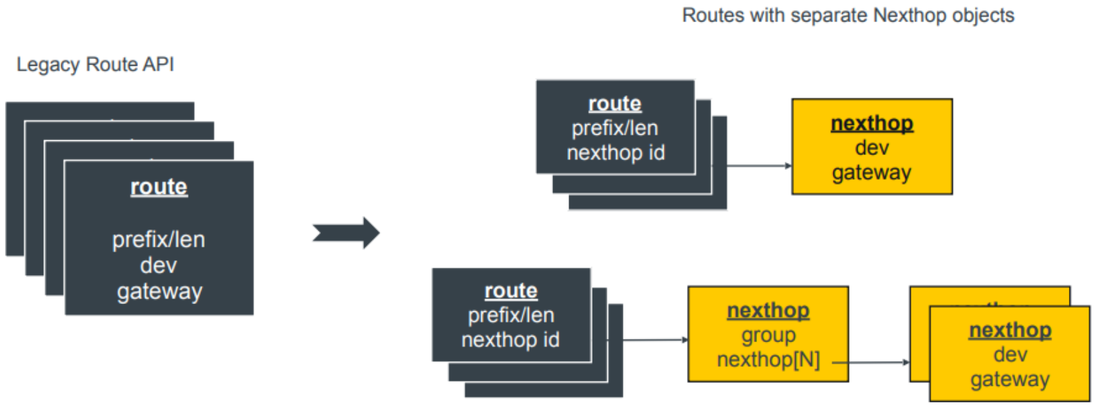


MOTIVATION – LOAD BALANCING



NEXTHOP OBJECTS OVERVIEW

- Introduced in 5.3 [1] by David Ahern
- Nexthop objects are managed separately from routes, unlike legacy API [2]



[1] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=ab84be7e54fc3d9b248285f1a14067558d858819
[2] https://www.linuxplumbersconf.com/event/4/contributions/434/attachments/251/436/nexthop-objects-talk.pdf

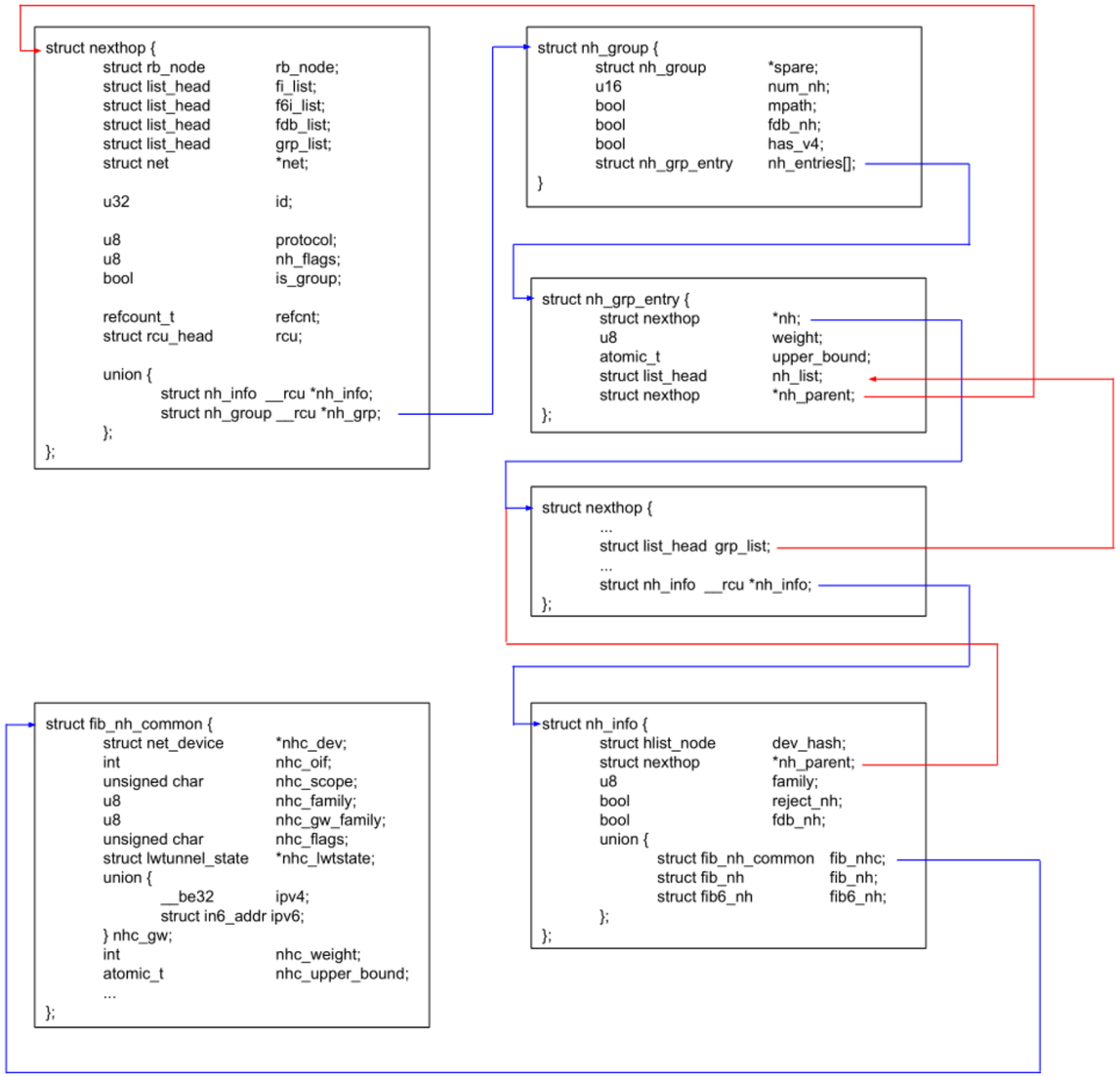
NVIDIA.

NEXTHOP OBJECTS – INTERFACE OVERVIEW

- **Add two single nexthops**
```
# ip nexthop add id 1 via 192.0.2.2 dev dummy1
# ip nexthop add id 2 via 198.51.100.2 dev dummy2
```
- **Create a group from the two nexthops**
```
# ip nexthop add id 10 group 1/2
```
- **Change the weights of the two nexthops to 3 and 5**
```
# ip nexthop replace id 10 group 1,3/2,5
```
- **Retrieve info about the group**
```
# ip nexthop show id 10
id 10 group 1,3/2,5
```
- **Point a route to the group**
```
# ip route add 192.0.2.0/24 nhid 10
```
- Netlink interface: RTM_NEWNEXTHOP, RTM_DELNEXTHOP, RTM_GETNEXTHOP

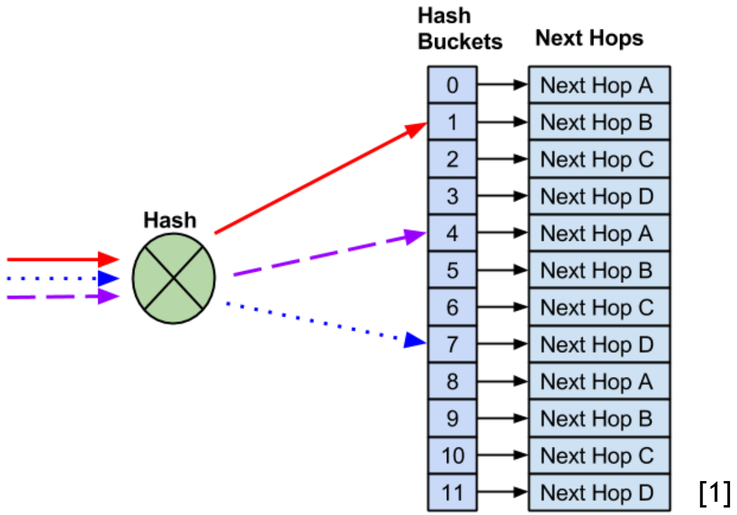
NVIDIA.

NEXTHOP OBJECTS – KERNEL DATA STRUCTURES



RESILIENT NEXTHOP GROUPS - THEORY

- Hash space is divided into a **fixed** number of nexthop buckets
- Number of nexthop buckets >> number of logical nexthops
- `NH = buckets[hash % #buckets].NH`

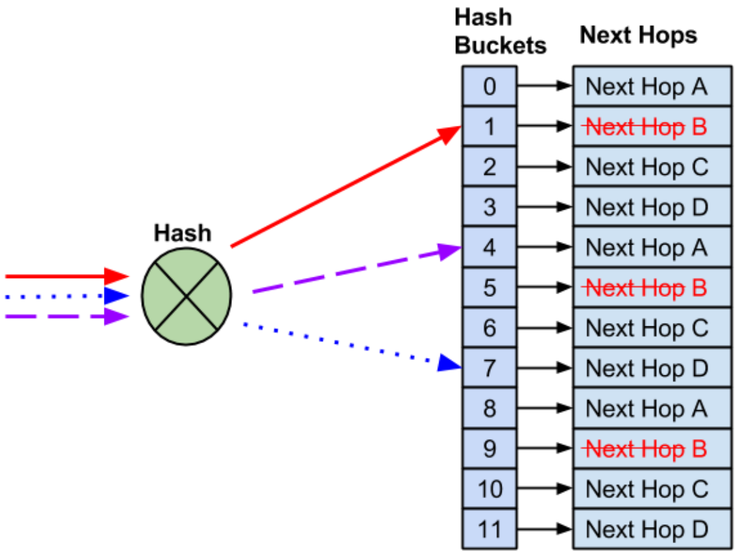


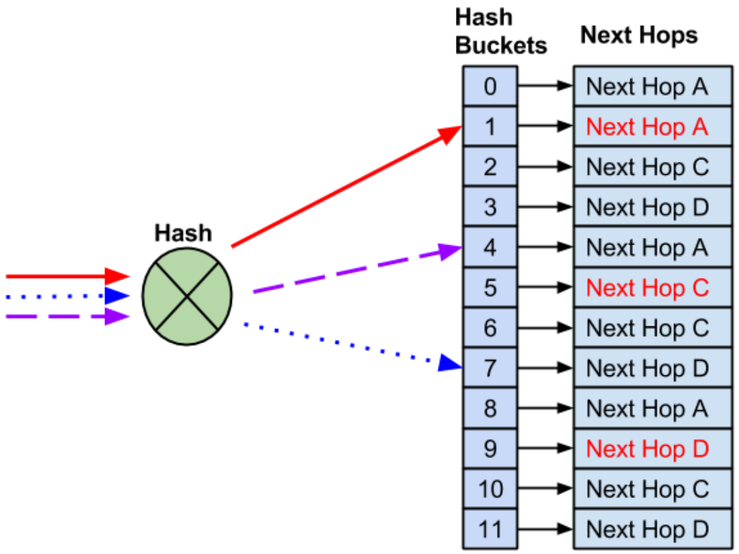
[1]

[1] https://docs.nvidia.com/networking-ethernet-software/cumulus-linux-44/Layer-3/Routing/Equal-Cost-Multipath-Load-Sharing-Hardware-ECMP/

RESILIENT NEXTHOP GROUPS - NEXTHOP REMOVAL

- Number of buckets does not change
- Only flows to removed nexthops are affected. The rest are not

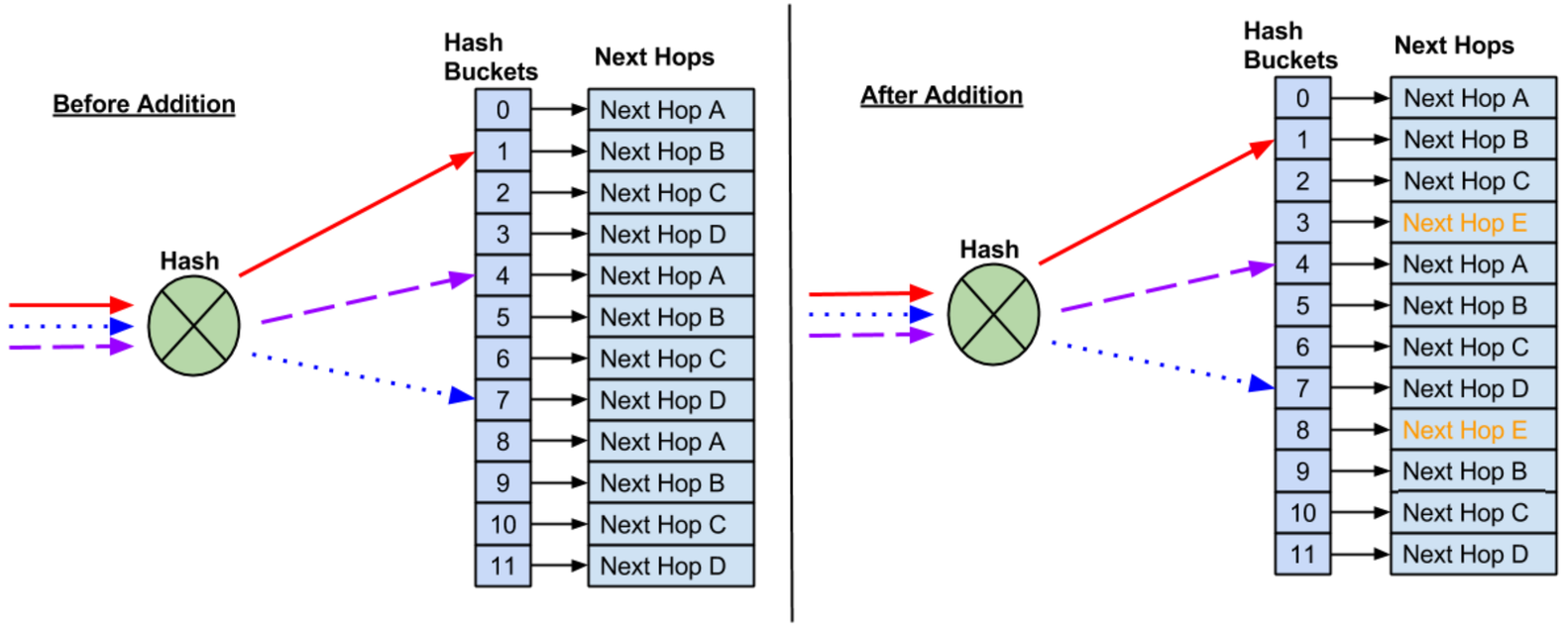




RESILIENT NEXTHOP GROUPS - NEXTHOP ADDITION

- Some buckets need to be remapped to the new nexthop
- The new nexthop might not be added immediately
- Idle buckets are chosen to avoid breaking existing connections
- Active buckets are only used when no idle buckets are available
- Two knobs
  - Idle timer: Time in seconds in which a nexthop bucket does not see traffic and is therefore considered idle
  - Unbalanced timer: Time in seconds in which a nexthop group is unbalanced and is therefore considered unbalanced. After which rebalancing will occur, which might result in some flows being reset

RESILIENT NEXTHOP GROUPS - NEXTHOP ADDITION (CONT.)



RESILIENT NEXTHOP GROUPS – INTERFACE OVERVIEW

- **Add two single nexthops**

```
# ip nexthop add id 1 via 192.0.2.2 dev dummy1
# ip nexthop add id 2 via 198.51.100.2 dev dummy2
```

- **Create a resilient group from the two nexthops**

```
# ip nexthop add id 10 group 1/2 type resilient buckets 8 idle_timer 120 unbalanced_timer 300
```

- **Change attributes of the group**

```
# ip nexthop replace id 10 group 1/2 type resilient idle_timer 100
# ip nexthop replace id 10 group 1/2 type resilient unbalanced_timer 900
# ip nexthop replace id 10 group 1,9/2,11 type resilient
```

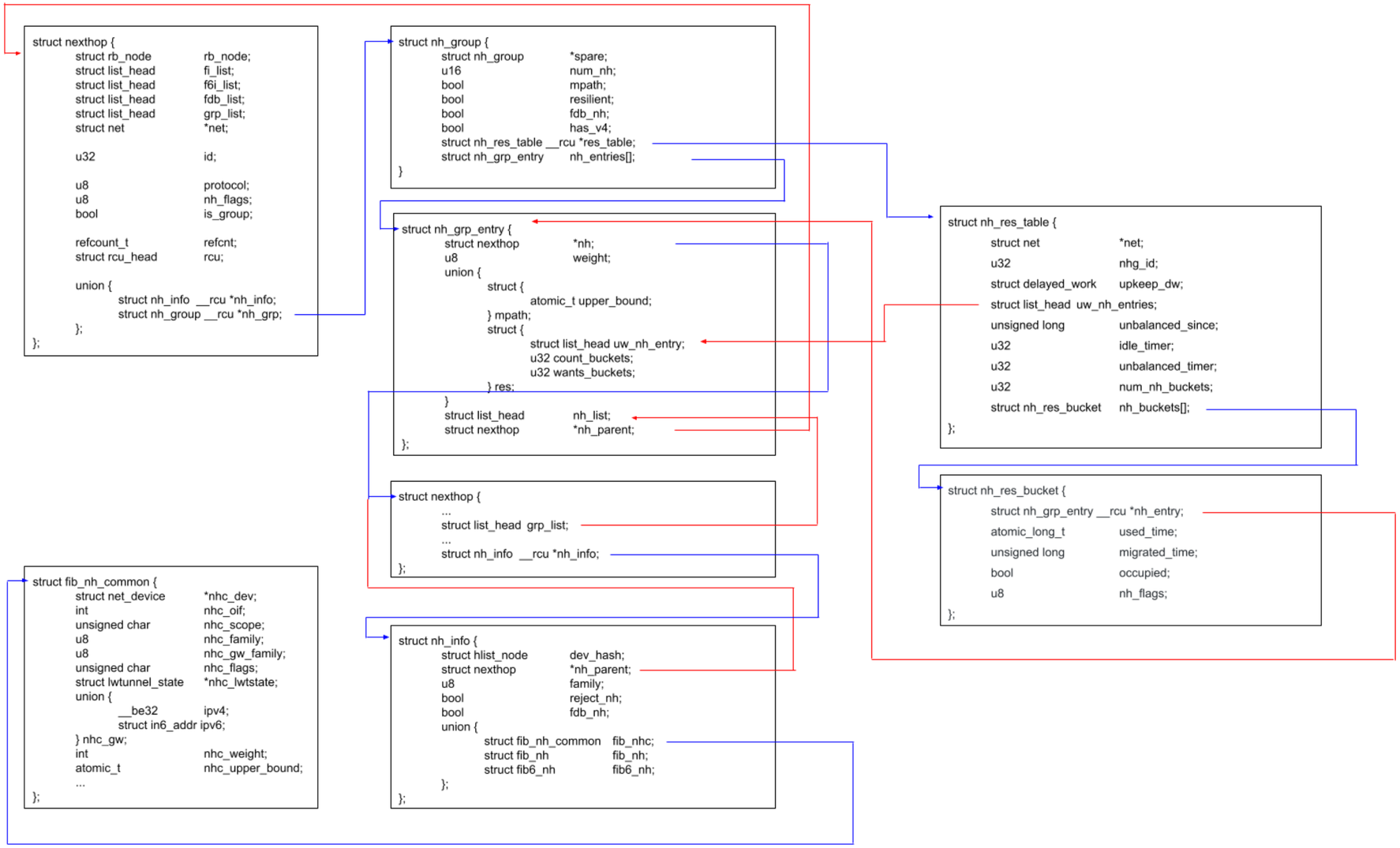
- Netlink interface: RTM_NEWNEXTHOP, RTM_DELNEXTHOP, RTM_GETNEXTHOP, RTM_GETNEXTHOPBUCKET

INTROSPECTION

- ```
  # ip nexthop show id 10
  id 10 group 1/2 type resilient buckets 8 idle_timer 60 unbalanced_timer 300 unbalanced_time 0
  ```

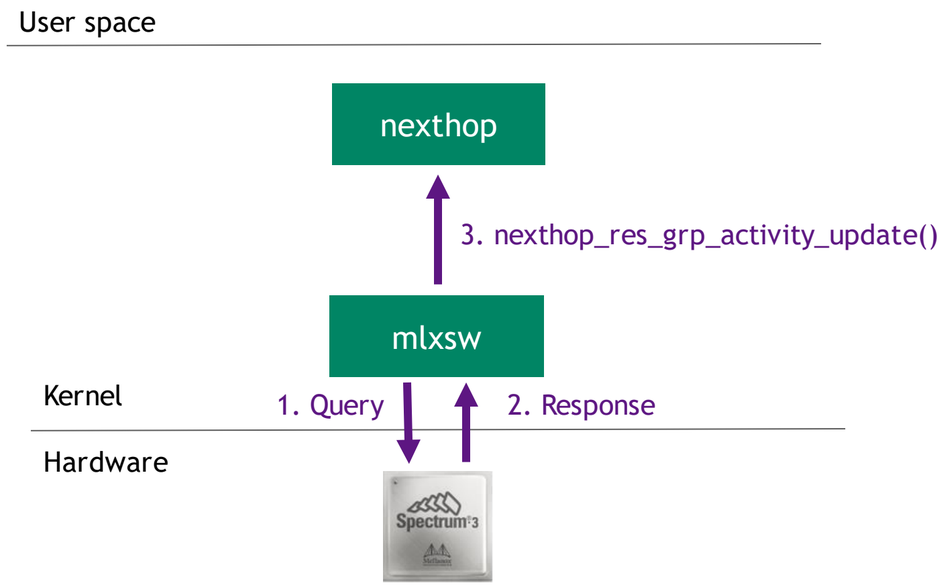
- ```
  # ip nexthop bucket show id 10
  id 10 index 0 idle_time 5.59 nhid 1
  id 10 index 1 idle_time 5.59 nhid 1
  id 10 index 2 idle_time 8.74 nhid 1
  id 10 index 3 idle_time 8.74 nhid 1
  id 10 index 4 idle_time 8.74 nhid 2
  id 10 index 5 idle_time 8.74 nhid 2
  id 10 index 6 idle_time 8.74 nhid 2
  id 10 index 7 idle_time 8.74 nhid 2
  ```

KERNEL DATA STRUCTURES



HARDWARE OFFLOAD

- Nexthop bucket activity / idleness is a key parameter to ensure resiliency
- Kernel updates "used time" of nexthop bucket when forwarding packets
- When forwarding is offloaded to hardware, device drivers need to report activity to the kernel



TESTING

- Lots of new tests were added [1]
- 49 new API tests under `tools/testing/selftests/net/fib_nexthops.sh`
- IPv4/IPv6 forwarding tests under `tools/testing/selftests/net/forwarding/router_mpath_nh_res.sh`
- GRE forwarding tests under `tools/testing/selftests/net/forwarding/gre_multipath_nh_res.sh`
- TODO: Describe new netlink attributes (e.g., `NHA_RES_GROUP`) and new messages (e.g., `RTM_GETNEXTHOPBUCKET`) to syzkaller. Basic attributes already described [2]

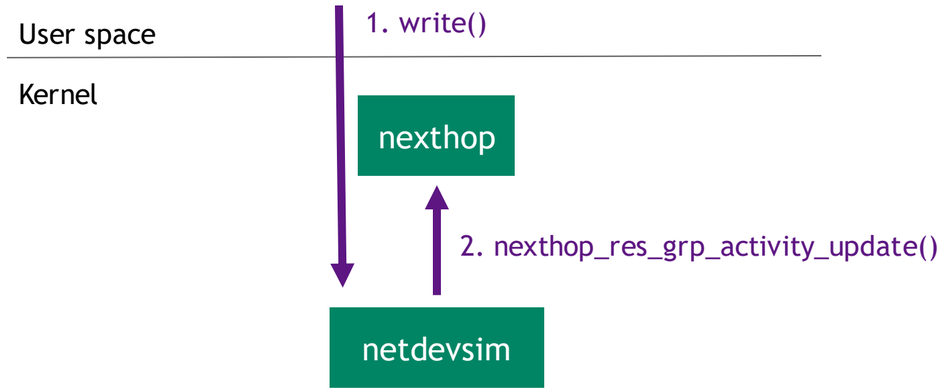
[1] https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=e9e90a70cc2d5690b84c7ca2f604e09a85222bb1
[2] https://github.com/google/syzkaller/commit/cb9c120a74bbfc083572692616096a4ee5ad11cc

NVIDIA.

TESTING (CONT)

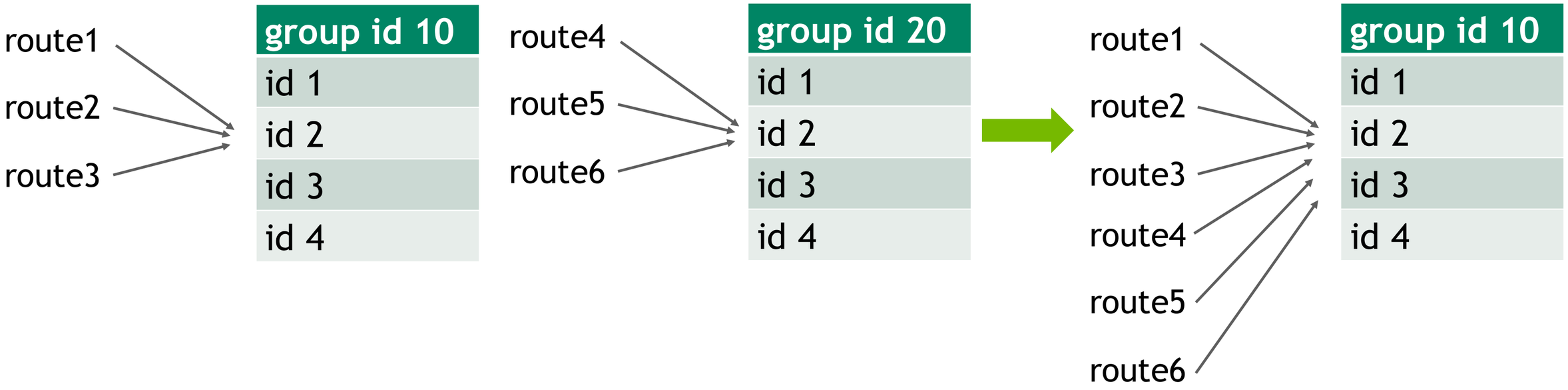
- Hardware offload API tests under tools/testing/selftests/drivers/net/netdevsim/nexthop.sh
- Full control over nexthop bucket activity allows for exhaustive tests of resilient nexthop groups
- Enabled by introducing new debugfs knob to netdevsim, which implements dummy offload of nexthop groups
- Mark bucket 23 in nexthop group 10 as active:

```
# echo "10 23" > /sys/kernel/debug/netdevsim/netdevsim10/fib/nexthop_bucket_active
```



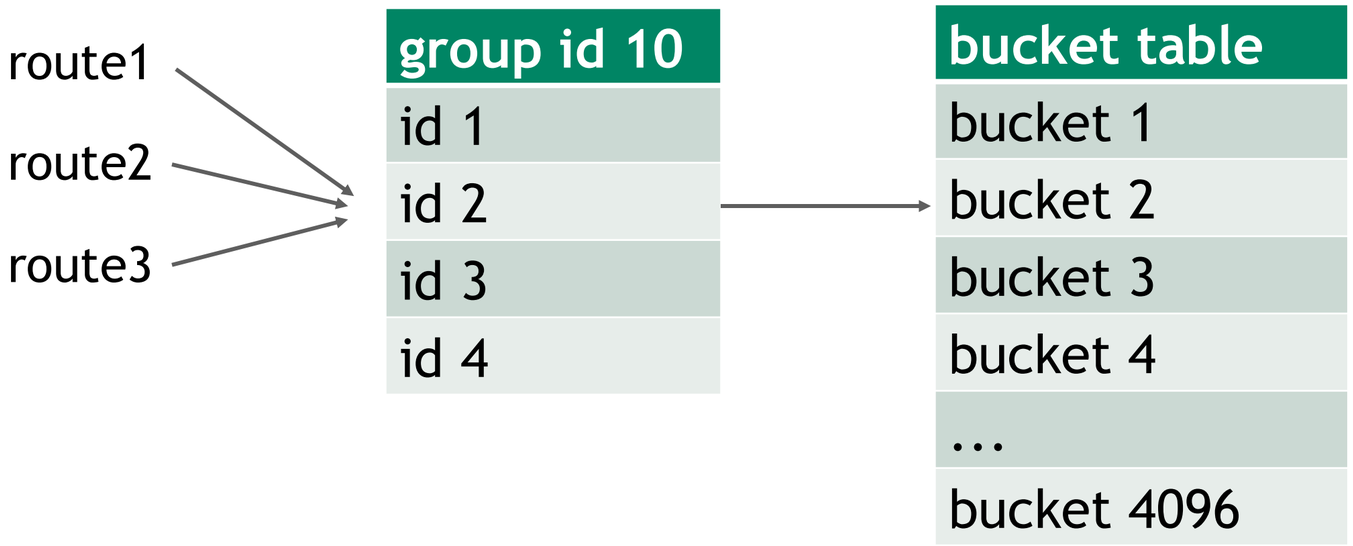
FUTURE EXTENSIONS – GROUP CONSOLIDATION

- Reduce resource usage by consolidating identical nexthop groups



FUTURE EXTENSIONS – GROUP CONSOLIDATION (CONT)

- Problem is even more severe with resilient nexthop groups due to large number of buckets

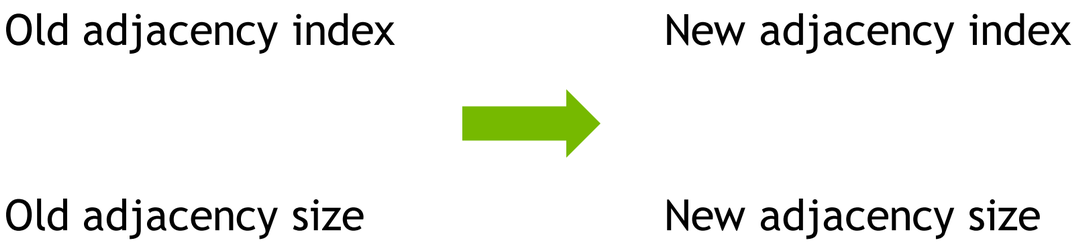


- Routing daemons should only consolidate groups that are identical at the bucket level

NVIDIA.

FUTURE EXTENSIONS – GROUP CONSOLIDATION (CONT)

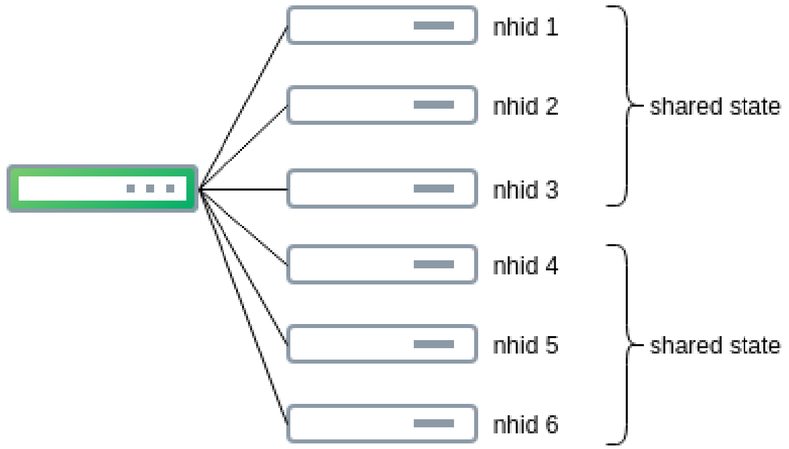
- Currently, consolidation can result in millions of route replacements
- Kernel already knows which routes use which nexthop object
- **Walk all routes pointing to nexthop object 10 and point them to nexthop object 20**
  `# ip nexthop replace id 10 nhid 20`
- **Delete old nexthop object**
  `# ip nexthop del id 10`
- Switch ASICs are built to perform such replacements atomically [1]:



[1] https://elixir.bootlin.com/linux/v5.13.1/source/drivers/net/ethernet/mellanox/mlxsw/reg.h#L7761

FUTURE EXTENSIONS – STATIC RESILIENT NEXTHOP GROUPS

- Nexthop objects are dynamically mapped by the kernel to nexthop buckets
- Therefore, a nexthop bucket can transition from any nexthop object to any nexthop object
- Problematic when state is shared between a subset of nexthop

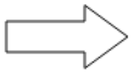


FUTURE EXTENSIONS – STATIC RESILIENT NEXTHOP GROUPS (CONT)

- Connection hashed to a nexthop bucket can transition from one subset to another

#	bucket table
0	nhid 1
1	nhid 2
2	nhid 3
3	nhid 4
4	nhid 5
5	nhid 6
6	nhid 1
7	nhid 2
8	nhid 3
9	nhid 4
10	nhid 5
11	nhid 6

nhid 1 goes down



#	bucket table
0	nhid 4
1	nhid 2
2	nhid 3
3	nhid 4
4	nhid 5
5	nhid 6
6	nhid 5
7	nhid 2
8	nhid 3
9	nhid 4
10	nhid 5
11	nhid 6

NVIDIA.

FUTURE EXTENSIONS – STATIC RESILIENT NEXTHOP GROUPS (CONT)

- Introduce static mode where user space can determine mapping of nexthop objects to nexthop buckets
- Allow user space to "color" nexthop buckets by assigning them to banks
- When a nexthop goes down, the kernel will re-populate nexthop buckets according to their bank

#	bucket table
0	nhid 1 (bank 1)
1	nhid 2 (bank 1)
2	nhid 3 (bank 1)
3	nhid 4 (bank 2)
4	nhid 5 (bank 2)
5	nhid 6 (bank 2)
6	nhid 1 (bank 1)
7	nhid 2 (bank 1)
8	nhid 3 (bank 1)
9	nhid 4 (bank 2)
10	nhid 5 (bank 2)
11	nhid 6 (bank 2)



#	bucket table
0	nhid 2 (bank 1)
1	nhid 2 (bank 1)
2	nhid 3 (bank 1)
3	nhid 4 (bank 2)
4	nhid 5 (bank 2)
5	nhid 6 (bank 2)
6	nhid 3 (bank 1)
7	nhid 2 (bank 1)
8	nhid 3 (bank 1)
9	nhid 4 (bank 2)
10	nhid 5 (bank 2)
11	nhid 6 (bank 2)

FUTURE EXTENSIONS – STATIC RESILIENT NEXTHOP GROUPS (CONT)

- Configuration example:

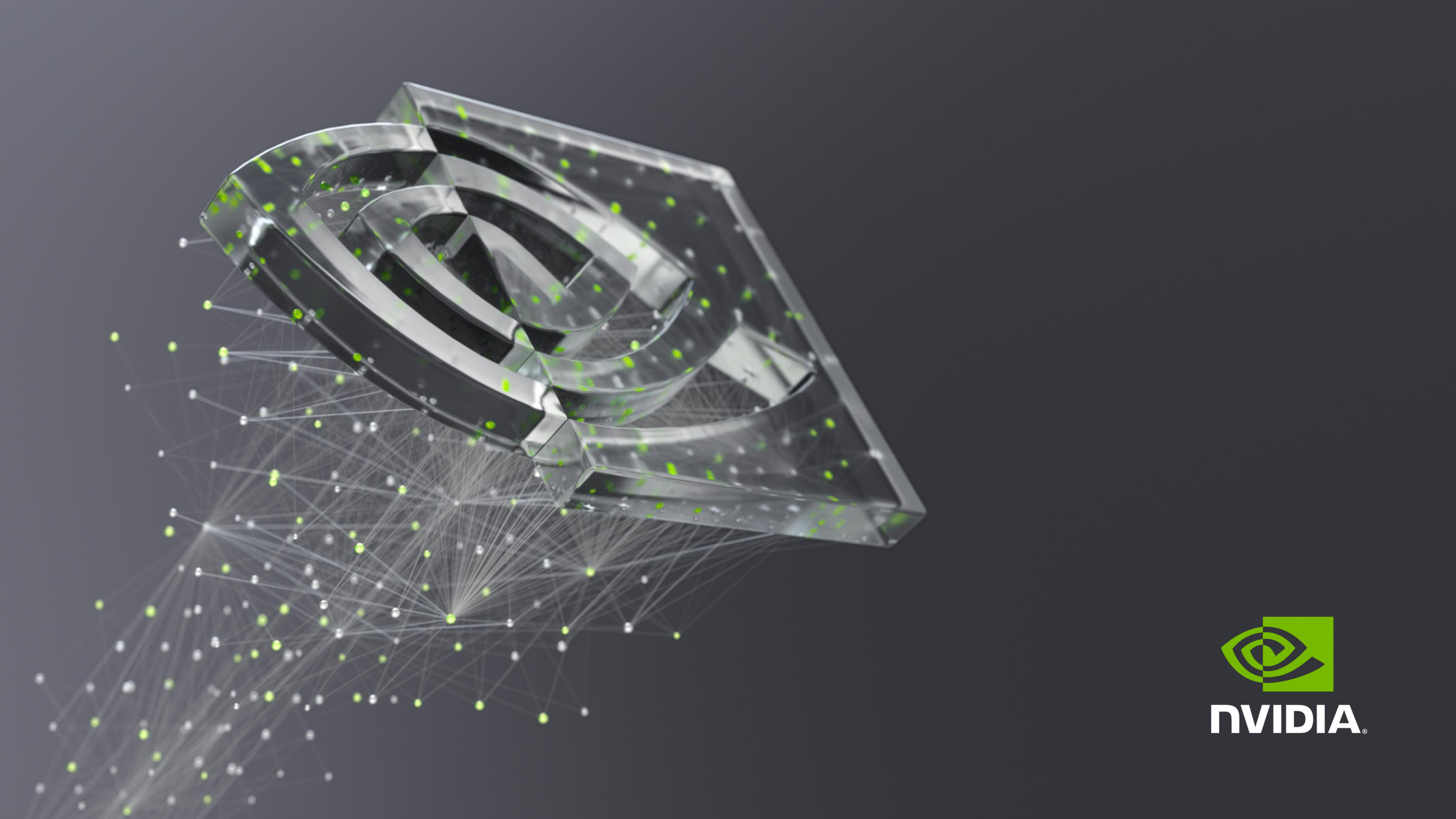
```
# ip nexthop add id 10 group 1/2/3/4/5/6 type resilient buckets 12 mode static
# ip nexthop bucket replace id 10 bucket 0 nhid 1 bank 1
# ip nexthop bucket replace id 10 bucket 1 nhid 2 bank 1
# ip nexthop bucket replace id 10 bucket 2 nhid 3 bank 1
# ip nexthop bucket replace id 10 bucket 3 nhid 4 bank 2
# ip nexthop bucket replace id 10 bucket 4 nhid 5 bank 2
# ip nexthop bucket replace id 10 bucket 5 nhid 6 bank 2
# ip nexthop bucket replace id 10 bucket 6 nhid 1 bank 1
# ip nexthop bucket replace id 10 bucket 7 nhid 2 bank 1
# ip nexthop bucket replace id 10 bucket 8 nhid 3 bank 1
# ip nexthop bucket replace id 10 bucket 9 nhid 4 bank 2
# ip nexthop bucket replace id 10 bucket 10 nhid 5 bank 2
# ip nexthop bucket replace id 10 bucket 11 nhid 6 bank 2
```

FUTURE EXTENSIONS – FRR SUPPORT

- FRR supports nexthop objects since version 7.3 [1]
- Adding support for resilient nexthop objects should be straightforward
- All the heavy lifting is done by the kernel

[1] https://github.com/FRRouting/frr/commit/c0038fca9b4b736a59d6ed786c12ac0c8a2909ac

NVIDIA.


NVIDIA